UD 01 · ANEXO PRÁCTICO 05

Python aplicado a la empresa

Primera programación en Python desde cero · AulaTec

- Primeros scripts
- Odoo vía API
- CSV importable

Introducción a Python aplicada a AulaTec

En Odoo puedes crear contactos, buscar clientes y preparar presupuestos mediante formularios. En esta carpeta empezarás a escribir programas propios que realizan operaciones empresariales sencillas. Empezarás con scripts locales y terminarás consultando los contactos reales de tu Odoo mediante Python, además de crear un CSV que podrás importar desde su interfaz.

Qué vas a conseguir

Preparar Python en Windows; ejecutar un archivo .py; utilizar print(), variables, números y cadenas; leer datos con input(); convertirlos con int() y float(); calcular importes y presentar resultados con f-strings.

1. Preparar Python en Windows

- Descarga Python 3 desde https://www.python.org/downloads/ y ejecuta el instalador para Windows. Marca «Add Python to PATH» si aparece esa opción.
- Abre una terminal nueva (PowerShell). Ejecuta uno de los comandos de comprobación que se muestran a continuación. Si Windows ofrece abrir la tienda en lugar de Python, comprueba la instalación y los alias de ejecución de aplicaciones.

```
py -3 --version
python --version
```

Una de las órdenes debe mostrar una versión de Python 3. Las capturas y los mensajes de la instalación pueden variar ligeramente. No confundas la terminal de Python, que muestra >>>, con PowerShell, donde se ejecutan los archivos.

2. Crear y ejecutar tu primer archivo

Crea una carpeta para tus prácticas, por ejemplo C:\SGE\Python_UD1. Abre esa carpeta con un editor de texto o un editor de código. Guarda un archivo llamado hola_empresa.py con este contenido:

```
print("Bienvenido a AulaTec")
```

Abre PowerShell en esa misma carpeta y ejecuta uno de estos comandos (usa el que funcione en tu equipo):

```
py -3 hola_empresa.py
# Alternativa:
python hola_empresa.py
```

El programa debe mostrar «Bienvenido a AulaTec». Un archivo .py es un fichero de texto con instrucciones Python. Si escribes py -3 desde una terminal, arrancas el intérprete; si escribes py -3 seguido del nombre del archivo, ejecutas ese script.

3. Instrucciones, comentarios y orden de ejecución

Python lee el archivo y ejecuta sus instrucciones en orden. El símbolo # sirve para escribir un comentario que Python no ejecuta. print() muestra texto u otros valores en la terminal.

```
# Mensaje inicial de la aplicación
print("Empresa: AulaTec")
print("Área: atención al cliente")
```

4. Variables y tipos de datos

Una variable es un nombre que permite referirse a un valor. Se asigna con =. En Python no se escribe int, String o double antes del nombre de la variable: el objeto al que apunta determina el tipo del valor.

```
empresa = "AulaTec"       # str: cadena de texto
unidades = 3              # int: número entero
precio = 45.50            # float: número decimal
disponible = True         # bool: verdadero o falso
print(empresa)
print(unidades)
```

Las cadenas se escriben entre comillas; los decimales usan punto, no coma. True y False se escriben con mayúscula inicial. Los nombres pueden contener letras, números y _; no empiezan por número ni contienen espacios.

5. Cálculos con datos de una empresa

Los operadores +, -, *, / y ** permiten sumar, restar, multiplicar, dividir y elevar a una potencia. Para calcular el importe de una venta se multiplica el precio unitario por la cantidad.

```
precio = 45.50
cantidad = 3
subtotal = precio * cantidad
print(subtotal)
```

En el ejemplo, subtotal vale 136.5. No escribas «45,50» dentro del código: en Python una coma tiene otros usos. Para estos primeros ejercicios utilizaremos operaciones sencillas con float. En aplicaciones económicas reales suele emplearse una representación decimal apropiada, como Decimal.

6. Mostrar variables dentro de un mensaje

Una f-string es una cadena que empieza por f: su contenido entre llaves { } se evalúa y se incorpora al texto. También permite mostrar un número con dos decimales mediante :.2f.

```
cliente = "Lucía Vega"
importe = 136.5
print(f"Cliente: {cliente}")
print(f"Importe: {importe:.2f} EUR")
```

La salida muestra «Importe: 136.50 EUR». Mostrar dos decimales no cambia cómo se ha calculado el número; solo cambia su presentación.

7. Leer información del teclado con input()

input() muestra un mensaje y espera a que el usuario escriba un dato. El valor recibido siempre es una cadena de texto (str), aunque se tecleen cifras.

```
cliente = input("Nombre del cliente: ")
producto = input("Producto solicitado: ")
print(f"{cliente} solicita {producto}.")
```

8. Convertir texto a números

int() transforma una cadena que representa un entero, como «3», en un valor numérico entero. float() transforma una cadena como «45.50» en un número decimal. Si la entrada no tiene el formato adecuado, Python produce un error; aprenderás a validarla en la siguiente unidad.

```
cantidad_texto = input("Cantidad: ")
precio_texto = input("Precio unitario: ")
cantidad = int(cantidad_texto)
precio = float(precio_texto)
subtotal = cantidad * precio
print(f"Subtotal: {subtotal:.2f} EUR")
```

Al introducir un decimal utiliza un punto: escribe 45.50 y no 45,50. La conversión a número debe hacerse antes del cálculo. Si multiplicas una cadena por un entero, no obtendrás el importe de una venta.

9. Crear presupuestos de práctica

Un presupuesto sencillo combina datos de texto y cálculos: nombre del cliente, producto, precio unitario y cantidad. En las prácticas se usa un porcentaje fijo del 21 % únicamente como dato matemático del ejercicio; no representa una configuración fiscal de Odoo ni determina el impuesto aplicable a una operación real.

```
cliente = "Lucía Vega"
producto = "Teclado"
precio = 25.00
cantidad = 2
subtotal = precio * cantidad
impuesto = subtotal * 0.21
total = subtotal + impuesto
print(f"Cliente: {cliente} | Producto: {producto}")
print(f"Subtotal: {subtotal:.2f} EUR")
print(f"Total simulado: {total:.2f} EUR")
```

10. Errores habituales antes de continuar

- Si aparece SyntaxError, revisa las comillas, los paréntesis y que cada línea esté completa.
- Si aparece NameError, comprueba que la variable existe y que su nombre coincide exactamente, incluidas mayúsculas y minúsculas.
- Si aparece ValueError al convertir un dato, revisa que el número sea válido y que el separador decimal sea un punto.
- Si la terminal indica que no encuentra el archivo, entra en su carpeta o ejecuta el archivo indicando su ruta completa.

11. Primera conexión de Python con Odoo

Odoo guarda los contactos que creaste en la aplicación Contactos. Puedes consultarlos desde otro programa mediante su API externa. En este primer contacto utilizarás XML-RPC, un protocolo de comunicación compatible con Python. El módulo xmlrpc.client ya viene con Python: no necesita instalarse con pip.

Para conectarte necesitas la dirección de tu Odoo local (habitualmente http://localhost:8070; si no cambiaste el puerto, http://localhost:8069), el nombre de la base de datos que creaste en el navegador, tu correo o nombre de inicio de sesión y tu contraseña de Odoo. La contraseña maestra de creación de bases de datos NO es la contraseña del usuario.

La consulta se hará sobre el modelo res.partner (contactos). Un modelo representa una clase de datos en Odoo. La operación search_count devuelve cuántos registros puede consultar tu usuario; search_read recupera información de los registros. Estas consultas solo leen datos: no crean, actualizan ni eliminan contactos.

```
# Esquema simplificado de una consulta real a Odoo:
# conectarse -> autenticar -> consultar res.partner
#              -> mostrar el resultado en Python
```

La respuesta de search_read es una lista de diccionarios: cada diccionario contiene campos de un contacto, por ejemplo name y email. for sirve para recorrerlos; aprenderás a utilizar estas estructuras con más detalle en la siguiente unidad. Aquí se explican y se emplean con un ejemplo completo.

12. Enviar datos de Python hacia Odoo usando CSV

También puedes generar un fichero CSV desde Python y utilizar la función Importar registros del módulo Contactos. CSV significa valores separados por comas: cada fila es un contacto y cada columna es un campo. El ejemplo 04 utilizará csv.DictWriter para generar nombres, correos y teléfonos ficticios, comprobar el archivo y después importarlo desde Odoo.

Este procedimiento NO usa la API para escribir datos: Python crea un fichero y tú confirmas la importación en Odoo. Así aprenderás dos formas reales de intercambiar información: consulta directa de la API e importación de archivos.

13. Relación con Odoo y continuidad del curso

Comenzarás con scripts locales para mostrar, introducir y calcular datos. Después utilizarás Python para consultar contactos reales de Odoo mediante XML-RPC y para crear un CSV importable en su interfaz. En la UD2 profundizarás en decisiones, bucles, listas y funciones; en la UD3 desarrollarás módulos propios de Odoo.

Itinerario de esta carpeta

Primero realiza los ejemplos 01 y 02 y las relaciones 01 y 02 para aprender Python. Después completa los ejemplos 03 y 04 y las relaciones 03 y 04: ahí trabajarás con datos de tu propia instancia de Odoo.